

AN2DL - First Challenge Report

Neural Whales

Lorenzo Della Matera, Giovanni Ferola, Irene Ferrente, Greta Fornaciari

lorenzodella, gioferola, ireneferrente, gretafornciari

274104, 271423, 286342, 280128

November 17, 2025

1 Introduction

The objective of this challenge was to predict the **true pain status** of subjects, classifying them into one of three categories: **no_pain**, **low_pain**, or **high_pain**. This classification task relied on the analysis of multivariate time-series data captured over repeated observations.

1.1 Approach Overview

We approached this *timeseries classification* problem by doing these steps:

Data Preprocessing: Cleaning the raw time series data and structuring it using a sliding window approach

Model Selection: Evaluating and training appropriate machine learning models for multivariate time-series classification

Model Evaluation: Assessing performance using standard classification metrics to determine the **most accurate predictor**.

The input features included:

Temporal movement patterns (e.g., body joints coordinates).

Subject characteristics (static data)

Self-Reported Pain Scores (via the `pain_survey` features).

2.2 Main Challenges

The primary difficulties encountered were:

- Effectively managing the **class imbalance** to prevent model bias towards the majority class;
- Developing a model robust enough to capture subtle temporal patterns without **overfitting**.

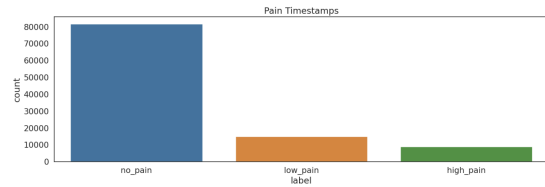


Figure 1: Dataset class unbalance

2 Problem Analysis

2.1 Dataset characteristics

The dataset consists of a collection of multivariate time series, where each `sample_index` represents an independent subject recording of 160 time steps.

3 Methods

3.1 Data preprocessing

The first thing we did was to extract the sequences made of 160 time steps for each sample index and

we gave them a unique id. It was also needed to convert **categorical** features into integer values, in order to be processed by the network, to do that we used label encoding. The same thing was done both for output classes and for subject characteristics.

Then we **normalized** all the numerical features in the dataset using min-max.

We also noticed that in the dataset the column `joint_30` contained only the values 0.5, so we decided to not consider it during the model training.

To prepare the data for model input, we first split the time series into smaller subsequences using a **sliding window**. This window moved across the sequence at a constant interval defined by the stride. In this way, each sample will be of size (`window_size`, `number_of_features`).

3.2 Model characteristics

Our model is a flexible class `RecurrentClassifier`, which contains two main modules:

- `self.rnn`: configurable recurrent block to use different net architecture (RNN, LSTM, GRU), different number of layers and neurons, and different dropout rate.
- `self.classifier`: fully connected layer for final classification.

The advantage of having a configurable net is that we could use it to perform a grid search to find the best parameters.

3.3 Training and Validation

To evaluate the model, we reserved 120 subjects for the validation set to ensure no data overlap between training and validation.

We trained the model using **Cross-Entropy Loss** combined with class weights (balanced) to handle the dataset imbalance. It gives adequate importance to the less frequent classes during training. Moreover, we implemented Early Stopping to stop training when the model stopped improving.

3.4 Regularization

Another crucial problem was **regularization**, a critical component in training deep learning models, that aims to mitigate overfitting. It helps the model generalize better by introducing constraints

or modifications during training. While testing various models, we noticed they tended to overfit quite easily. Even after adjusting **dropout rates** and **L1/L2** regularization parameters, the results showed only minor changes.

So, instead of trying to reduce overfitting, we decided to leverage it to our advantage and adopt a bagging approach.

3.5 Bagging

Bagging is an ensemble technique specifically chosen to mitigate the effects of overfitting and **enhance model robustness**. Since the Grid Search had identified several configurations with good performance, we opted to combine the three best models.

- First, we identified the best 3 models according to our grid search;
- We trained them using **bootstrap samples** (sampling with replacement) from the training set, ensuring diversity in the data on which the individual models learned;
- Finally, we combined their predictions using majority vote.

3.6 Submission generation

Once we were satisfied with the model's performance, we performed inference over the test set and generated a `.csv` file with the predictions.

Since each `sample_index` was segmented into multiple windows, our model produced multiple classifications per sequence. The final label was then determined by taking the mode of all its corresponding predictions.

4 Experiments

To identify the optimal model configuration, we conducted several experiments, with the key parameters and results summarized in Table 1.

4.1 Baseline

Our baseline (biLSTM V1) used a standard LSTM configured with a small window size and dropout rate, with no L1/L2 regularization. The F1 score over the validation set was quite high (0.923), serving as a performance benchmark.

Table 1: Main experiments using different parameters.

Model	Window	Stride	Learning Rate	Hidden Layers	Hidden Size	Dropout Rate	λ_1	λ_2	Batch Size	F1 val	F1 sub
biLSTM V1	20	5	1e-03	2	128	0.2	0	0	256	0.923	0.937
biLSTM V2	35	5	2e-03	2	128	0.4	1.3e-05	1.0e-03	128	0.931	0.957
biLSTM_bag	35	5	2e-03	2	128	0	0	0	128	0.9385	—
biRNN_bag	35	5	4e-04	1	256	0	0	0	128	0.9150	—
biGRU_bag	35	5	2e-03	2	128	0.2	0	0	256	0.9361	—

4.2 Grid Search

To improve the baseline, we initiated a **grid search** over the key hyperparameters shown in Table 1. We performed K-Fold Cross-Validation (K=3) over a search space that included parameter variations.

The best-performing configuration (biLSTM V2) was identified as a **Bidirectional LSTM**.

Once the best configuration was found, we retrained the model on the original training/validation split without K-Fold. We obtained a slightly better F1 score (0.931) with respect of the baseline.

4.3 Final Model

During the grid search, we found that many configurations provided good results, so we decided to use an ensemble method to combine their capabilities.

Since the models tended to overfit, we opted for **bagging**. In Table 1, we reported the best 3 models found in grid search. However, we discovered that setting all regularization parameters to 0 increased the F1 score of bagging (0.9375). The goal was to push the individual models towards overfitting, creating the ideal conditions for the bagging algorithm to **improve generalization performance**.

In Table 2, we see that bagging achieved better results than averaging the 3 models' performances.

Table 2: Comparison between the 3 base models and the **bagging models**.

Model	F1 val	F1 sub
biLSTM	0.9385	—
biRNN	0.9150	—
biGRU	0.9361	—
Average 3 models	0.9298	—
Bagging	0.9375 (+0.83%)	0.9664

5 Results

The experimental activity showed the effectiveness of both model optimization through Grid Search and the ensemble approach based on **Bagging**.

On the submission the biLSTM V2 achieved an F1-score of **0.957**, significantly improving over the baseline (0.937). However, the Bagging strategy showed even better performance, reaching an F1-score of **0.9664**, proving successful not only in terms of robustness but also in absolute performance on the final test set.

These results confirm that **combining different models via ensemble methods can outperform even the best single model**, improving generalization through the complementarity of architectures.

6 Discussion

The main strength of our approach is the **Bagging** ensemble strategy, which achieved strong generalization by leveraging the variance of the individual recurrent models. Also, we effectively mitigate class imbalance via **class weights**. However, the **Majority Vote** aggregation doesn't implement a tie-breaking system. This could lead to misclassifications due to models' inconsistencies.

Regarding our modeling assumptions, we decided to treat the subject characteristics as part of the time series, even if they did not vary over time.

7 Conclusions

In this project we successfully achieved a highly competitive result. However, more targeted regularization techniques and advanced **preprocessing** strategies could be explored to further enhance model performance and interpretability.